

Laporan
Penerapan *Object Detection* Menggunakan YOLOV11 dengan Dataset
Universe Roboflow

Untuk memenuhi tugas mata kuliah IK5127 - 01 Artificial Intelligence pada Sistem
Otonom



Disusun oleh:

Izma Alhazmi Herdian

13321027

PROGRAM STUDI TEKNIK FISIKA
FAKULTAS TEKNOLOGI INDUSTRI
INSTITUT TEKNOLOGI BANDUNG

2024

I. PENDAHULUAN

Pada era transformasi digital, penggunaan teknologi berbasis computer vision telah menjadi bagian penting dalam mendukung otomatisasi dan pemantauan lingkungan. Di Institut Teknologi Bandung (ITB), teknologi *object detection* memiliki potensi besar untuk diimplementasikan dalam berbagai keperluan, mulai dari peningkatan keamanan hingga penelitian di bidang *machine learning*. Laporan ini membahas penerapan YOLO versi 11 (terbaru), sebuah algoritma *object detection* yang dikenal karena kecepatan dan akurasinya, dalam mendeteksi objek di lingkungan kampus ITB.

Dalam penelitian ini, kami menggunakan dataset Universe Roboflow untuk mendeteksi beberapa objek spesifik yang umum dijumpai di lingkungan ITB, yaitu *person*, *car*, *fountain*, dan *tree*. Pemilihan objek ini mencerminkan kebutuhan untuk mengenali elemen-elemen penting dalam berbagai skenario kampus, seperti identifikasi kendaraan, pemantauan ruang hijau, dan pendeteksian orang dalam area tertentu.

Studi ini diharapkan dapat mengevaluasi performa YOLO v11 dalam lingkungan nyata dengan berbagai kondisi pencahayaan dan latar, sekaligus memberikan kontribusi terhadap penelitian dan pengembangan teknologi *object detection* di kampus. Selain itu, hasil penelitian ini diharapkan dapat memberikan pandangan praktis bagi pengembangan sistem otomatisasi dan pemantauan berbasis visual di lingkungan pendidikan di Indonesia.

II. MODEL

A. Dataset

Dataset yang digunakan dalam penelitian ini bersumber dari Universe Roboflow, khususnya dataset bernama *Al-Mustansiriya University Data Computer Vision Project*. Dataset ini dirancang untuk mendeteksi empat kelas objek, yaitu *car*, *person*, *fountain*, dan *tree*. Setiap objek dalam dataset telah dilabeli sesuai kelasnya, dengan format data yang sesuai untuk pelatihan pada model YOLO, yaitu *bounding boxes* untuk setiap objek dalam gambar.

Untuk memastikan konsistensi dan kesesuaian dataset dengan model YOLO v11, dilakukan beberapa proses *preprocessing*. Pertama, gambar mengalami proses *Auto-Orient*, yang secara otomatis mengatur orientasi gambar agar seragam. Kedua, gambar di-*resize* dengan ukuran 1280x720 piksel menggunakan teknik stretch untuk menjaga keseragaman dimensi input ke dalam model. Proses ini penting untuk mengurangi ketidakseragaman dimensi gambar dan membantu mempercepat waktu komputasi saat pelatihan model.

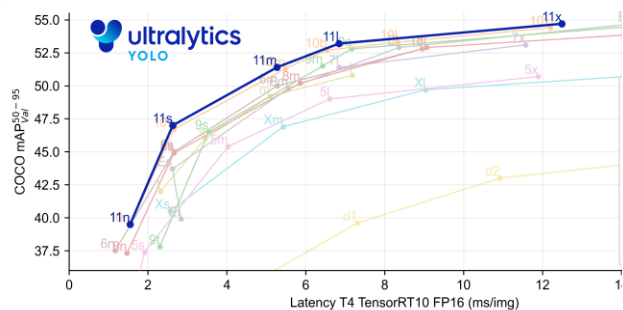
Dataset ini dibagi menjadi tiga bagian untuk memastikan kinerja model yang optimal: 604 gambar untuk set pelatihan, 76 gambar untuk set validasi, dan 75 gambar untuk set uji. Pembagian ini memberikan proporsi yang mendukung model dalam mempelajari pola dari data yang cukup banyak (sekitar 70% untuk pelatihan), sementara 15% masing-masing digunakan untuk validasi dan pengujian. Pembagian ini bertujuan untuk memastikan model tidak *overfitting* pada data

pelatihan, memungkinkan evaluasi kinerja model pada data yang belum pernah dilihat, serta menilai generalisasi model secara efektif.

Pada dataset ini, tidak diterapkan teknik augmentasi data seperti rotasi, *flipping*, atau *scaling*. Meskipun augmentasi umumnya digunakan untuk meningkatkan variasi data dan membantu model lebih *robust*, keputusan untuk tidak melakukan augmentasi dapat diambil berdasarkan kebutuhan khusus atau keterbatasan spesifik dari proyek ini, dengan asumsi bahwa variasi dalam dataset sudah cukup untuk mencapai performa deteksi yang memadai.

B. Implementasi Model

YOLO11, versi terbaru dari keluarga model YOLO yang dikembangkan oleh Ultralytics, dipilih karena keunggulannya dalam keseimbangan antara kecepatan dan akurasi. Model ini memiliki peningkatan arsitektural yang signifikan dibandingkan versi sebelumnya, sehingga dapat digunakan untuk deteksi objek secara *real-time* dengan efisiensi tinggi dan waktu latensi yang rendah. Dengan kemampuan ini, YOLO11 sangat cocok untuk aplikasi yang memerlukan deteksi objek secara cepat dan akurat.



Gambar 1 Yolo Versi 11

YOLO versi 11 merupakan keluarga model deteksi objek terbaru dari Ultralytics, yang mencakup beberapa varian dengan perbedaan performa dalam akurasi, kecepatan inferensi, jumlah parameter, dan kebutuhan komputasi. Setiap varian dilatih pada dataset COCO, yang mencakup 80 kelas objek. Model-model ini terdiri dari YOLO11n, YOLO11s, YOLO11m, YOLO11l, dan YOLO11x. Peningkatan ukuran model dari "n" ke "x" umumnya menghasilkan akurasi yang lebih tinggi, namun juga meningkatkan waktu inferensi, jumlah parameter, dan kebutuhan komputasi (FLOPs). Secara keseluruhan, model YOLO11 menyediakan fleksibilitas bagi pengguna untuk memilih antara akurasi yang tinggi dan efisiensi komputasi yang optimal sesuai dengan kebutuhan aplikasi.

Dalam tugas ini digunakan YOLO11n, varian ini merupakan yang paling ringan dan tercepat di antara keluarga YOLO11. Model ini memiliki ukuran input 640 piksel dan mencapai *mean Average Precision* (mAP) 50-95 sebesar 39.5%, yang merupakan tingkat akurasi yang cukup baik untuk aplikasi yang mengutamakan kecepatan. Kecepatan inferensi YOLO11n sangat tinggi, yaitu 1.5 ms per gambar pada TensorRT10 dan 56.1 ms pada CPU (ONNX). Dengan hanya 2.6 juta parameter dan kebutuhan komputasi sebesar 6.5 miliar FLOPs, model ini sangat efisien secara komputasi dan cocok untuk perangkat dengan keterbatasan sumber daya atau aplikasi yang membutuhkan deteksi objek *real-time*. YOLO11n menawarkan keseimbangan yang optimal antara kecepatan dan akurasi, meskipun dengan tingkat presisi yang lebih rendah dibandingkan varian YOLO11 lainnya.

Untuk melatih model, dataset yang terdiri dari empat kelas objek, yaitu car, person, fountain, dan tree, digunakan tanpa modifikasi signifikan pada arsitektur dasar YOLO v11. Beberapa penyesuaian dilakukan pada file konfigurasi YAML yang berisi informasi mengenai dataset, pembagian data pelatihan, validasi, dan pengujian, serta kelas-kelas objek yang akan dideteksi. Proses fine-tuning dilakukan dengan mengunggah dataset yang telah dibagi menjadi tiga set (train, validation, dan test) dalam format ZIP ke Google Colab, kemudian mengekstraknya untuk mempercepat proses pengunggahan. Model dilatih menggunakan hyperparameter yang disesuaikan, termasuk learning rate dan batch size, dengan 20 epoch sebagai jumlah iterasi pelatihan. Optimisasi dilakukan menggunakan algoritma default yang disediakan oleh YOLO v11, yang memastikan proses pelatihan yang cepat dan stabil.

Pelatihan model dilakukan dengan memanggil perintah `!yolo train model=yolo11n.pt data=data.yaml epochs=20 imgsz=640` di Google Colab, sehingga model ini dapat memproses dataset dan melakukan deteksi objek dengan ukuran gambar 640x640 piksel.

III. IMPLEMENTASI

A. Deteksi Objek *Real-Time*

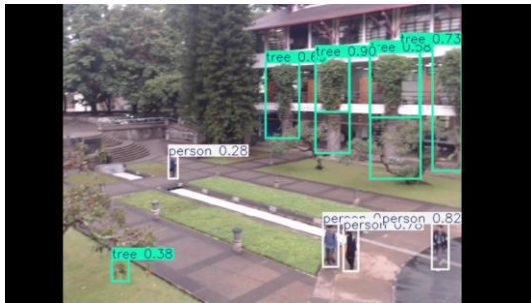
Pada implementasi deteksi objek *real-time*, model YOLO v11 yang telah dilatih di Google Colab diterapkan untuk mendeteksi objek secara langsung di lingkungan sekitar ITB. Uji deteksi dilakukan di VS Code Jupyter, yang digunakan untuk membuka

webcam dan mengambil data video secara langsung.

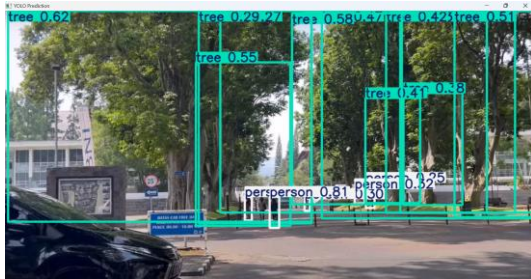
Perangkat keras yang digunakan dalam implementasi ini adalah laptop dengan webcam internal yang memiliki resolusi tinggi, sehingga dapat merekam video dengan kualitas baik untuk deteksi objek. Webcam digunakan sebagai sumber data untuk aplikasi deteksi objek. Selain itu, video yang telah diunduh dan foto-foto juga digunakan sebagai sumber data tambahan untuk pengujian.

Proses deteksi objek dimulai dengan memproses setiap frame video yang direkam dari webcam. Setiap frame dianalisis oleh model YOLO v11 yang telah dilatih untuk mendeteksi objek-objek seperti car, motorcycle, person, dan tree. Ketika objek terdeteksi, model menggambar bounding box di sekitar objek tersebut dan menampilkan label objek berserta confidence score yang menunjukkan tingkat kepastian model terhadap deteksi tersebut.

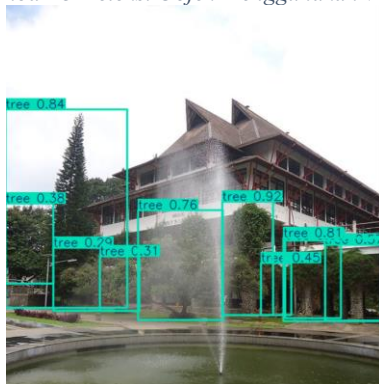
Aplikasi deteksi objek ini juga dapat menerima data dalam tiga bentuk: pengambilan data secara *real-time* menggunakan *webcam*, menggunakan video yang sudah diunduh, dan menggunakan foto yang diunduh. Teknik ini memastikan deteksi objek dapat dilakukan dalam berbagai skenario dan aplikasi langsung, seperti pemantauan lingkungan kampus atau sistem pengawasan berbasis visi komputer.



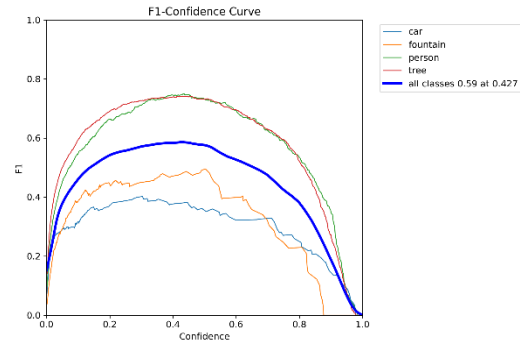
Gambar 2 Deteksi Object Menggunakan Webcam Secara Real-Time



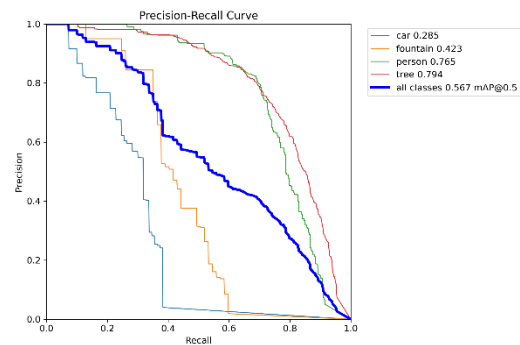
Gambar 3 Deteksi Objek Menggunakan Video



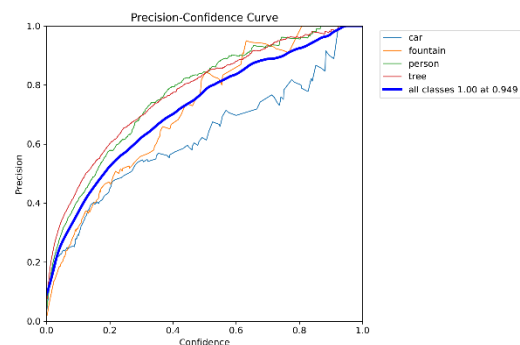
Gambar 4 Deteksi Objek Menggunakan Foto



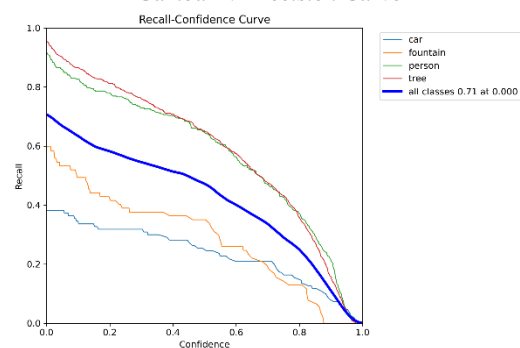
Gambar 5 F1 Curve



Gambar 6 Precision Recall Curve



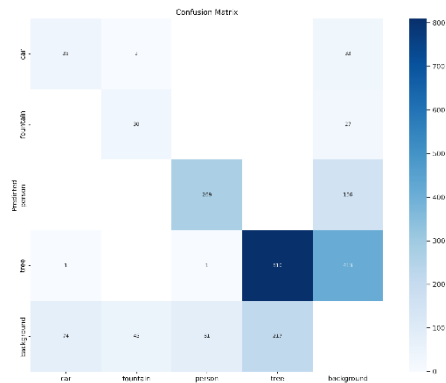
Gambar 7 Precision Curve



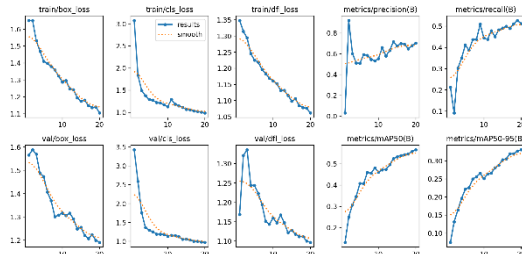
Gambar 8 Recall Curve

B. Evaluasi dan Hasil

Setelah model YOLO v11 dilatih menggunakan dataset yang telah disiapkan, tahap evaluasi kinerja model dilakukan untuk mengukur efektivitas model dalam mendeteksi objek secara akurat dan andal. Untuk mengevaluasi model ini, beberapa metrik yang umum digunakan dalam deteksi objek diterapkan, yaitu *Confusion Matrix*, *F1 Curve*, *Precision Curve*, *Precision-Recall Curve*, dan *Recall Curve*.



Gambar 9 Confusion Matrix



Gambar 10 Result

Dari matriks *confusion* menunjukkan kinerja model dalam mengklasifikasikan lima kategori: mobil, air mancur, orang, pohon, dan latar belakang. Model memiliki tingkat keberhasilan yang bervariasi dalam setiap kategori. Misalnya, objek mobil berhasil diklasifikasikan dengan benar sebanyak 35 kali, tetapi sering salah diklasifikasikan sebagai orang (33 kali) dan latar belakang (74 kali), menunjukkan bahwa model terkadang kesulitan membedakan mobil dari orang atau elemen latar belakang. Untuk kategori air mancur, model mengklasifikasikan dengan benar 27 kali, tetapi sering salah mengenali air mancur sebagai mobil (30 kali) dan latar belakang (45 kali). Pada kategori orang, model berhasil mengklasifikasikan 269 kali dengan benar, namun terdapat kesalahan besar di mana objek orang salah dikenali sebagai latar belakang sebanyak 81 kali. Kategori pohon memiliki performa terbaik

dengan 810 klasifikasi benar, tetapi masih terdapat kesalahan di mana pohon salah dikenali sebagai latar belakang sebanyak 413 kali. Terakhir, kategori latar belakang teridentifikasi benar sebanyak 217 kali, namun kerap salah dikenali sebagai pohon (413 kali) dan orang (156 kali). Kesalahan terbesar terjadi pada objek-objek yang mirip secara visual, seperti antara pohon dan latar belakang, serta orang dan latar belakang. Untuk meningkatkan akurasi, model dapat dilatih ulang dengan lebih banyak data yang memperkuat perbedaan antara kategori-kategori yang sering salah dikenali ini.

C. Analisis Kesalahan

Berdasarkan matriks kebingungan dari model deteksi objek yang telah dilatih, terdapat beberapa jenis kesalahan klasifikasi yang memengaruhi kinerja model. Kesalahan-kesalahan ini terbagi dalam dua kategori utama, yaitu false positive dan false negative. False positive terjadi ketika model mendeteksi keberadaan objek yang sebenarnya tidak ada, seperti pada kasus latar belakang yang sering salah dikenali sebagai pohon atau orang. Di sisi lain, false negative terjadi ketika model gagal mendeteksi objek yang ada dalam gambar, seperti kategori mobil yang beberapa kali salah diklasifikasikan sebagai latar belakang. Hanya ada 35 prediksi benar untuk mobil, sementara banyak prediksi salah di kategori ini, yang mengindikasikan kelemahan model dalam mengenali mobil.

Beberapa faktor diduga menjadi penyebab utama terjadinya kesalahan klasifikasi ini. Pertama, ukuran dataset yang terbatas, yaitu hanya terdiri dari 755 gambar, dan proses

pelatihan yang dilakukan dengan jumlah epoch sebanyak 20, mungkin belum cukup untuk model mempelajari karakteristik visual setiap objek dengan baik. Dengan jumlah epoch yang relatif rendah, model kemungkinan belum mencapai konvergensi yang optimal, sehingga kemampuan dalam mengenali objek masih terbatas. Faktor kedua adalah kemiripan visual antar objek, seperti antara pohon dan air mancur, yang menyebabkan kesalahan klasifikasi pada kedua kategori ini, terutama dalam lingkungan yang kompleks.

Kategori latar belakang juga mengalami kesalahan klasifikasi yang signifikan. Beberapa prediksi latar belakang salah dikenali sebagai pohon (217 kasus) dan orang (81 kasus). Hal ini mungkin disebabkan oleh kemiripan fitur visual antara latar belakang dengan objek-objek lain atau akibat kompleksitas latar dalam gambar yang digunakan untuk pelatihan. Selain itu, tampaknya model belum mampu mengidentifikasi perbedaan antar objek dalam kondisi visual yang beragam, seperti variasi ukuran, sudut pandang, atau pencahayaan.

Untuk meningkatkan performa model deteksi objek ini, beberapa langkah perbaikan dapat dilakukan. Pertama, menambah variasi data pada kategori mobil, baik dari segi ukuran maupun sudut pandang, akan membantu model mempelajari karakteristik mobil dengan lebih baik. Penerapan teknik augmentasi data, seperti rotasi, perubahan skala, dan variasi pencahayaan, juga disarankan agar model menjadi lebih tahan terhadap perubahan lingkungan dan latar

belakang. Kedua, peningkatan jumlah epoch dalam proses pelatihan dapat membantu model mempelajari pola antar kategori dengan lebih mendalam, sehingga akurasi deteksi dapat ditingkatkan. Ketiga, optimasi hyperparameter, khususnya learning rate, dapat membantu model mencapai kinerja yang lebih stabil, mengurangi tingkat kesalahan deteksi, dan secara keseluruhan meningkatkan akurasi.

IV. KESIMPULAN

Penerapan model YOLO v11 pada dataset Al-Mustansiriya University Data Computer Vision Project telah berhasil mendeteksi objek-objek yang ada dalam gambar, terutama untuk objek person dan tree. Model menunjukkan performa yang baik dalam mendeteksi kedua objek ini, dengan hasil deteksi yang cukup akurat. Namun, untuk objek car dan fountain, performa model masih kurang optimal. Hal ini disebabkan oleh jumlah data yang sangat terbatas untuk kedua kelas objek tersebut dalam dataset, yang mempengaruhi kemampuan model dalam mendeteksi objek-objek tersebut dengan baik.

Untuk meningkatkan akurasi deteksi, khususnya pada kelas car dan fountain, perlu dilakukan penambahan jumlah data pada kedua kelas tersebut. Selain itu, model juga dapat ditingkatkan performanya dengan memperpanjang jumlah epoch pada proses pelatihan. Dengan menambah epoch, model akan memiliki lebih banyak waktu untuk belajar dan mengoptimalkan bobot-bobotnya, sehingga diharapkan dapat menghasilkan deteksi yang lebih akurat pada

objek yang memiliki representasi data lebih sedikit.

Secara keseluruhan, meskipun model sudah memberikan hasil yang memadai untuk objek person dan tree, masih terdapat potensi untuk perbaikan, terutama pada objek-objek dengan jumlah data yang lebih terbatas. Oleh karena itu, peningkatan jumlah data dan pelatihan yang lebih lama akan sangat berpengaruh terhadap peningkatan performa deteksi model ini.

LAMPIRAN

```
%pip install ultralytics
!git clone
https://github.com/ultralytics/yolov11
%pip install -qr requirements.txt

%cd yolov11

import torch
import ultralytics
ultralytics.checks()
```

```
!unzip "Yoloww.zip"
```

```
# Train YOLO11n
!yolo train model=yolo11n.pt
data=data.yaml epochs=20 imgsz=640
```

```
import cv2
from ultralytics import YOLO

# Load the model
model = YOLO('best.pt') # Update path to
where you saved the model locally

# Run real-time detection
cap = cv2.VideoCapture(0)
while True:
    ret, frame = cap.read()
    if not ret:
        break
```

```
results = model(frame)
annotated_frame = results[0].plot()

cv2.imshow('YOLO Real-Time Detection',
annotated_frame)
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
```

```
import cv2
from ultralytics import YOLO

# Load model yang sudah dilatih (best.pt)
model = YOLO('best.pt') # Pastikan best.pt
adalah path yang benar untuk model Anda

# Load foto yang sudah diunduh (misalnya
foto.jpg)
image = cv2.imread('fda.jpg') # Ganti
dengan path file gambar Anda

# Lakukan prediksi pada gambar
results = model(image)

# Menggambar bounding boxes dan label pada
gambar
annotated_image = results[0].plot()

# Tampilkan gambar dengan hasil prediksi
cv2.imshow("YOLO Prediction",
annotated_image)

# Tunggu hingga pengguna menekan tombol
untuk menutup
cv2.waitKey(0)
cv2.destroyAllWindows()
```